

Radial-Layered Procedural Dungeon Generation

Prof. Proloy Biswas

Dept. of Computer Science Engineering
Dayananda Sagar University
Devarakagallahalli, Harohalli,
Kanakapura, Bangalore 562112
proloybiswas-cse@dsu.edu.in
<https://orcid.org/0000-0002-8350-9034>

Tejas KM

Dept. of Computer Science Engineering
Dayananda Sagar University
Devarakagallahalli, Harohalli,
Kanakapura, Bangalore 562112
eng22cs0480@dsu.edu.in
<https://orcid.org/0009-0001-9313-0159>

Sriram Puranik

Dept. of Computer Science Engineering
Dayananda Sagar University
Devarakagallahalli, Harohalli,
Kanakapura, Bangalore 562112
eng22cs0467@dsu.edu.in
<https://orcid.org/0009-0007-4668-5411>

Subrath Shrotriya

Dept. of Computer Science Engineering
Dayananda Sagar University
Devarakagallahalli, Harohalli,
Kanakapura, Bangalore 562112
eng22cs0471@dsu.edu.in
<https://orcid.org/0009-0002-7520-4262>

Kalyan Gogoi

Dept. of Computer Science Engineering
Dayananda Sagar University
Devarakagallahalli, Harohalli,
Kanakapura, Bangalore 562112
eng23cs1031@dsu.edu.in
<https://orcid.org/0009-0002-0085-6039>

Abstract—Making complex and enjoyable game levels can be easily done using procedural dungeon generation. Usually, traditional approaches use charts or straight lines which can prevent designers from coming up with fresh levels. In this paper, we introduce a new Radial-Layered Procedural Dungeon Generation algorithm that groups rooms on concentric rings stemming from a central point. Such flexibility makes this method suitable for various ways to play and for games in both 2D and 3D formats. Allowing users arrange rooms across the map as they please and use different themes, our system breaks with traditional dungeon generation patterns. We present the algorithm's design, the adjustable values it requires and we prove its effectiveness with several practical combinations.

Keywords—Procedural Content Generation, Adaptive Dungeon Layout, 2D and 3D Game Design, Room Placement Flexibility, Randomized Algorithms

I. INTRODUCTION

Modern games rely on Procedural Content Generation (PCG) to ensure their environments are engaging, can be repeated many times and are resource-saving. Because Procedural Dungeon Generation (PDG) ends up with endless level combinations, it keeps players engaged without needing to endlessly create new levels. For this problem, methods such as Binary Space Partitioning (BSP), Cellular Automata and Random Walk algorithms are widely used. Generating useful rooms with these techniques sometimes leads to built work that uses lots of right angles. Consequently, many of these randomly made dungeons resemble numbers instead of sculpted rooms, lacking their own story and distinctive designs. In this novel, we explain how we designed Radial-Layered Procedural Dungeon Generation which lays dungeon geometry out around a central origin in concentric rings. Every ring serves as a layer of the dungeon, giving designers the chance to programming changes in difficulty, types of areas or special meaning right into the map. Rooms are distributed along the spoke-like bands which provides a plan that supports both future circulation and direction from the center outwards. It allows for flexible

exploration of different paths and displays interesting visual patterns rare in current experiments on procedural generation.

This method can perform faster than other approaches from a computer science perspective. Thanks to a direct transformation from polar to Cartesian coordinates, the placement of rooms is done in $O(n)$ time and does not need complex or recursive algorithms during runtime.

Here Fig.1, we explain the fundamental algorithm, describe its use in both 2D and 3D contexts, and show how radial-layered layouts improve performance and enhance the aesthetic appearance of the environment. Moreover, we explain how this technique is more efficient and controllable than common methods, giving rise to fresh ideas in procedurally created locations.

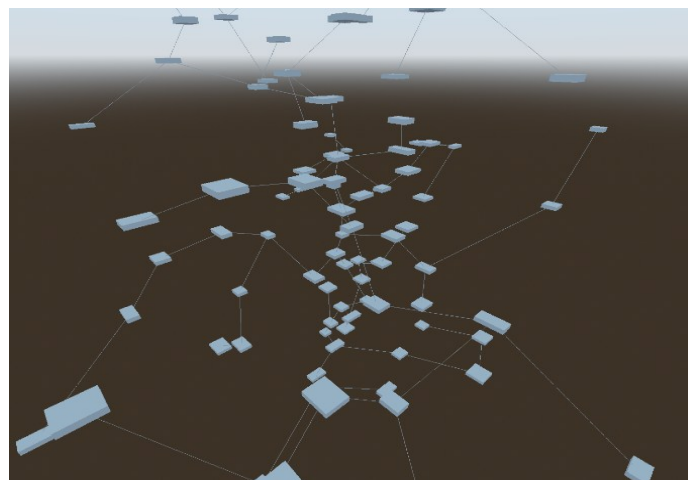


Fig. 1. Radial Layered Procedural Dungeon Layout

II. LITERATURE SURVEY

Procedural Content Generation (PCG) has become a foundation in modern game development, allowing developers to create repeatable, and computationally efficient environments. This technique is particularly common in the generation of dungeons, where the goal is to produce levels that are both engaging and structurally diverse. This section examines important earlier procedural dungeon generation research and places our Radial-Layered method in this developing field.

Early methods in dungeon generation often utilized space partitioning or cellular systems. Binary Space Partitioning (BSP) and Random Walk algorithms, while effective, frequently produced grids of rectangular rooms with right-angled hallways. Ashlock's work on automatic map generation [8] and the Tanagra system by Smith et al. [5] exemplify tools that, despite offering flexibility, often leaned on rectilinear and linear design paradigms. These layouts can limit aesthetic diversity and spatial storytelling.

Research by Hendrikx et al. [1] provided a comprehensive survey of PCG techniques, identifying major design patterns and challenges in content generation. Togelius et al. [2], and later Shaker et al. [3], expanded this analysis by discussing the goals and frameworks for procedural level design, emphasizing the trade-off between designer control and algorithmic novelty. Meanwhile, Dormans [11] and Compton & Mateas [9] focused on generating levels and missions in action-adventure and platform games, highlighting the importance of spatial narrative in design. Togelius and Schmidhuber [12] extended this further into automatic game design, exploring how generation itself can shape gameplay mechanics. Mechanic Miner explores how mechanics influence layout generation through reflection-driven design, relevant to our structure-aware approach [4]

Recent research has explored player-centered design and adaptive PCG. Yannakakis and Togelius [6] introduced the concept of experience-driven PCG, where player behavior influences level evolution. Summerville et al. [10] further explored this with PCG via machine learning (PCGML), enabling systems to learn level structure from data. In parallel, van der Linden et al. [14] and Mawhorter & Mateas [15] examined how procedural levels can support structured difficulty curves and story progression—goals that align well with our use of concentric radial layers. The radial layout also facilitates procedural storytelling, resonating with Hartevelt et al.'s emphasis on spatial narrative design [7].

Our approach diverges by introducing Radial-Layered Procedural Dungeon Generation, which organizes rooms along concentric rings from a central point. This geometric strategy, made efficient through polar-to-Cartesian coordinate mapping, supports $O(n)$ complexity for placement and enables both 2D and 3D extensibility. By focusing on structure and symmetry over randomness alone, our work aligns with calls for more controllable, yet expressive PCG systems [2][3][13].

In summary, while prior work has made important findings in both the automation and personalization of level design, few

have explored radial spatial layouts. Our contribution bridges this gap, offering a new algorithmic model that supports visual coherence, thematic layering, and runtime efficiency.

III. METHODOLOGY

A. Overview of the Generation Process

In this algorithm, Fig.2 Radial-Layered Procedural Dungeon Generation, rooms are arranged along circles that start from the middle and grow out in increasing circles. All rings form separate layers of the dungeon, allowing for both moving air and changing themes. It's easy to use sections of the process in different ways for games that use either 2D or 3D environments. There are three main phases in the methodology: creating the ring and the rooms, building the corridors and converting for 2D/3D use.

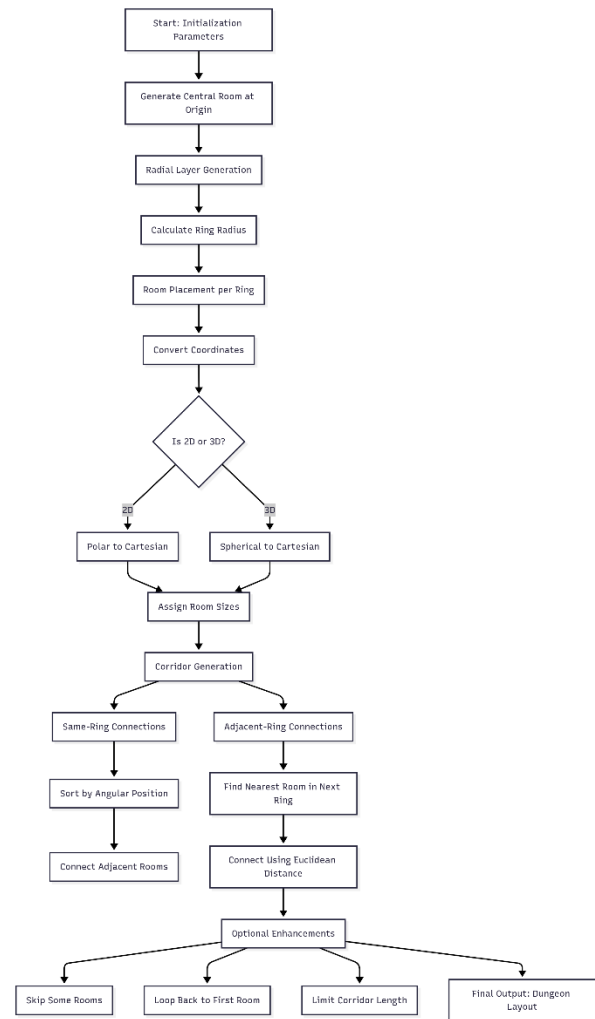


Fig. 2. Architecture diagram of the Radial-Layered Procedural Dungeon Generation algorithm.

At the start, the algorithm cuts up the available dungeon space into several rings, with each having a calculated radius. Rooms on each ring are grouped in evenly separated directions. Using polar-to-Cartesian (for planar) or spherical-to-Cartesian (for spatial) conversion techniques, the location of every room is set with reference to the center of the room. Following room

placement, a connector room is added to each side of the ring and corridors are added to link both the overlapping ring rooms and rooms on either side of them.

By using this approach, dungeons are made both appealing to look at and align with the rules of the game which supports a wide variety of events and stories. Direct equations in placement are reliable and consistent, so using them makes the algorithm fit for real-time game generation.

B. Ring and Room Placement

1) *Radial Layer (Ring) Generation*: The algorithm breaks down the dungeon into layers that resemble concentric spheres, or “rings,” radiating out from a central room at the origin.

Number of Rings: This is determined by the constant NUM_RINGS, which specifies how many of these layers will be created.

Ring Radius Calculation:

$$r_i = \left(\frac{(i+1) \cdot R_{\max}}{N} \right) + \text{randf_range}(-j, j) \quad (1)$$

where:

- r_i is the radius of the i^{th} ring,
- $R_{\max}$ is the maximum radius of the dungeon,
- N is the total number of rings,
- j is the ring jitter amount (i.e., the allowed randomness in ring radius).

This Fig.3 method enhances gameplay design. Each ring can be linked to specific challenges, providing intuitive progression. Scaling and randomness factors allow for customized designs.

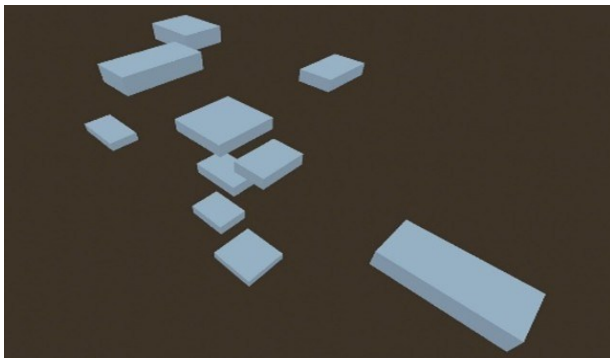


Fig. 3. Procedurally generated room layout using concentric rings. Rooms are arranged with even angular spacing plus jitter, vertically distributed via random polar angles for full spherical coverage, and assigned random sizes.

2) *Room Distribution per Ring*: Room Count per Ring:

Controlled by ROOMS_PER_RING, allowing flexibility.

Angle Calculation with Jitter:

$$\theta_i = \left(\frac{2\pi}{n_r} \right) \cdot i + \text{randf_range}(-a, a) \quad (2)$$

where:

- θ_i is the angular position of the i^{th} room on the ring,
- n_r is the number of rooms on the current ring,
- a is the angular jitter amount (i.e., random angular offset).

Vertical Distribution: In 3D, a random polar angle ϕ in $[0, \pi]$ allows full spherical coverage.

Spherical to Cartesian Conversion:

$$x = r \cdot \sin(\phi) \cdot \cos(\theta),$$

$$y = r \cdot \cos(\phi),$$

$$z = r \cdot \sin(\phi) \cdot \sin(\theta)$$

Room Size Range: Randomized within [MIN ROOM SIZE, MAX ROOM SIZE].

Center Room: Always placed at origin (0,0,0).

C. Corridor Generation

Creating corridors between rooms is a key part of designing a dungeon that is easy to navigate. This process can be broken down into two main stages: connecting rooms within the same radial ring and linking rooms between adjacent rings. The corridor generation algorithm ensures that all rooms are well connected, both within their own rings and across different layers of the dungeon.

1) *Same-Ring Connections*: For the rooms situated in the same ring, the basic approach is to connect each room with the one immediately next to it in a clockwise direction. This follows naturally from the circular layout of the rooms along the edge of the ring, where they are spaced evenly—with a slight angular perturbation to add some unpredictability.

Connecting neighbouring rooms is done straightforwardly by directly linking the positions of adjacent rooms via corridors. Circular Connection: The rooms are sorted by their angular position within the ring, and each room is connected to the next room in the sorted order. The last room of the ring is connected back to the first room, forming a closed loop so that all rooms within the same ring are reachable from each other.

In Fig.4 practice, the rooms in each ring are grouped by their ring index, and sorting is done based on angular position. Then, connections are established between every adjacent pair of rooms around the ring.

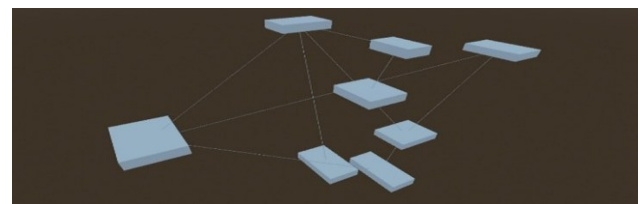


Fig. 4. Small-Scale 3D room cluster with corridor connections

2) *Adjacent-Ring Connections*: Corridors are generated between rooms in adjacent rings. The nearest-neighbor strategy is employed: a given room in one ring is connected to a room in an adjacent ring (either above or below) that has the minimum Euclidean distance from it.

To improve performance, spatial techniques such as angular bucketing or spatial partitioning may be used. These techniques reduce the number of rooms considered for each connection, thus saving computational time.

Once the nearest neighbors are identified, corridors are created to connect the two rooms, ensuring vertical connectivity and supporting the multi-layer structure of the dungeon.

3) *Optional Enhancements for Corridor Generation*: Additional enhancements may be applied by the developer to increase creative flexibility in dungeon design:

- **Skipping Rooms**: To create a more irregular layout, certain rooms can be skipped during connection. This results in a less predictable and more organic dungeon.
- **Looping Back**: Instead of connecting rooms linearly, the last room of each ring can be connected back to the first room, forming a closed loop for enhanced traversal.
- **Distance Cap**: A maximum connection radius can be enforced to avoid excessively long or impractical corridors.
- **Connection Limit**: Each room may be restricted to a maximum number of corridor connections. This constraint simplifies the overall dungeon graph and helps avoid overly dense connectivity.

These improvements give game designers better control over dungeon layout, allowing for more varied, complex, and thematic level designs. Depending on the selected features, developers can tune the dungeon's navigability and appearance to suit specific gameplay needs.

D. Algorithm Complexity

The computational complexity of the proposed radially-layered procedural dungeon generation algorithm is evaluated in terms of its two primary operations: room placement and room connection. Although the approach is similar for both 2D and 3D implementations, the latter involves additional spatial computations due to its volumetric nature.

1) Room Placement:

2D Implementation:

In 2D, each room's position is determined using polar coordinates:

$$x = r \cdot \cos(\theta),$$

$$y = r \cdot \sin(\theta)$$

All operations in this phase—radius/angle calculation, coordinate transformation, and room instantiation—are constant time per room. Let N be the total number of rooms:

Time Complexity (2D Placement): $O(N)$

3D Implementation:

In 3D, room positions are computed using spherical coordinates:

$$x = r \cdot \sin(\phi) \cdot \cos(\theta),$$

$$y = r \cdot \cos(\phi),$$

$$z = r \cdot \sin(\phi) \cdot \sin(\theta)$$

Similar to 2D, each step—coordinate conversion and positioning—is performed in constant time, yielding:

Time Complexity (3D Placement): $O(N)$

2) Room Connection:

2D Implementation:

Connections are established both within the same ring and between adjacent rings.

- *Same-Ring Connections*: Rooms are sorted by angular position and connected to their immediate neighbor. Naively comparing every pair leads to a worst-case cost of $O(k^2)$ per ring, where k is the number of rooms in that ring.
- *Adjacent-Ring Connections*: Each room connects to its nearest room in the next ring using Euclidean distance. Given N total rooms across R rings, the overall connection cost is:

Time Complexity (2D Connection):

$$O\left(\frac{N^2}{R}\right)$$

3D Implementation:

The 3D connection logic mirrors the 2D strategy but incorporates an additional spatial dimension in distance calculations.

- *Same-Ring*: Rooms are sorted by azimuth angle and connected sequentially, with optional looping to form closed rings.
- *Adjacent-Ring*: Nearest-neighbor search is performed via brute-force, as in 2D. The inclusion of the elevation angle and third coordinate increases only the constant factors.

Time Complexity (3D Connection):

$$O\left(\frac{N^2}{R}\right)$$

Despite the added dimensionality in 3D, the asymptotic complexity remains the same, ensuring scalability across different game types.

E. 2D and 3D Implementation Differences

The radial-layered dungeon generation algorithm is designed to support both two-dimensional (2D) and three-dimensional (3D) environments. While the core logic—placing

rooms in concentric layers around a central point—remains consistent, the implementations differ in spatial structure, coordinate transformations, room placement strategies, and connectivity.

1) *Room Placement in 2D*: In 2D, the dungeon is generated on a planar surface using polar coordinates. Each ring corresponds to a circle centered at the origin, and rooms are placed based on angular separation with optional jitter to avoid uniformity.

- **Radial Positioning**: Radii are calculated per ring with controlled randomness.
- **Coordinate Transformation**: Polar coordinates (r, θ) are converted to Cartesian coordinates (x, y) using standard trigonometric functions.
- **Room Size and Offset**: Rooms are assigned random sizes within specified bounds and offset to ensure proper centering.

This approach is well-suited for top-down dungeon designs, offering clear visual structure and symmetry.

2) *Room Placement in 3D*: In 3D, room positions are distributed over spherical shells instead of flat circles. The system uses spherical coordinates (r, θ, ϕ) , where θ is the azimuthal angle and ϕ is the elevation angle.

- **Spherical Distribution**: Each ring becomes a spherical shell, and rooms are assigned spherical coordinates for even spatial coverage.
- **Coordinate Transformation**: Positions are converted using:

$$x = r \cdot \sin(\phi) \cdot \cos(\theta)$$

$$y = r \cdot \cos(\phi)$$

$$z = r \cdot \sin(\phi) \cdot \sin(\theta)$$

- **Room Dimensions**: Rooms are modeled as 3D cuboids with random width and depth, and configurable height.

This enables rich, volumetric layouts with vertical layering and spatial complexity not possible in 2D.

3) *Corridor Construction and Spatial Connectivity*: The connection strategy is consistent in both 2D and 3D but adapts to the geometry of each space.

- **Same-Ring Connections**: Rooms are connected sequentially by angular (2D) or azimuthal (3D) order to form circular loops.
- **Adjacent-Ring Connections**: Each room connects to its nearest neighbor in the adjacent outer ring based on Euclidean distance.

In 2D, these paths are typically straight lines, while in 3D they span across all spatial axes, requiring additional consideration for elevation and intersection handling.

TABLE I: Comparison between 2D and 3D radial dungeon generation methods

Aspect	2D Generation	3D Generation
Coordinate System	Polar (r, θ)	Spherical (r, θ, ϕ)
Room Placement	x, y coordinates	x, y, z coordinates
Algorithm Complexity	Moderate (planar logic)	High (3D trigonometry)
Room Overlap Handling	Simple 2D check	Requires 3D collision detection

IV. RESULTS

To evaluate the efficiency of the proposed Radial-Layered Procedural Dungeon Generation algorithm, we conducted a series of tests in a 3D environment. The algorithm maintains a linear time complexity of $O(n)$ for both room placement and direct nearest-neighbor connection, where n is the total number of rooms.

TABLE II: Generation and connection times for radial algorithm

Total Rooms	Generation Time (ms)	Connection Time (ms)
10	0.50	0.34
80	3.47	3.39
200	6.92	7.88
500	11.21	14.24

As shown in Table 2, both generation and connection times scale linearly with the number of rooms, confirming the expected theoretical performance. The direct connection scheme, which uses angular sorting and Euclidean distance calculations, remains efficient even as room count increases.

We alternatively measured BSP-based dungeon generation using Prim's Minimum Spanning Tree (MST) algorithm for room connection. Although MST performs adequately for small graphs, it exhibits super linear growth, becoming impractical for larger dungeons.

TABLE III: Performance comparison with Prim's MST algorithm (ms)

Total Rooms	Generation Time	Prim's MST Connection Time
10	1.09	0.65
80	5.31	70.34
200	12.15	1203.52
500	23.86	9518.22

At 500 rooms, the native radial method completed the connection phase in 14.24 ms, compared to 9518.22 ms for Prim's MST. Even at moderate sizes (80 rooms), the radial method was faster.

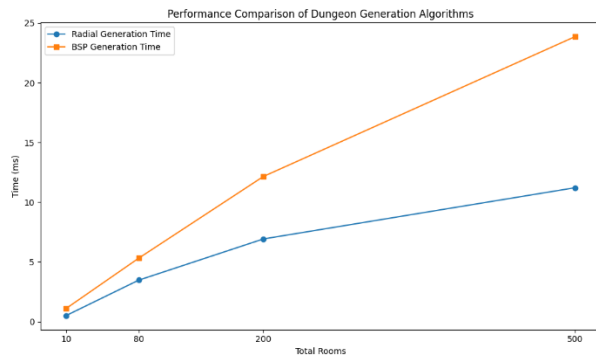


Fig. 5. Performance comparison of dungeon generation algorithms

These results Fig.5 highlight the scalability and practicality of the native radial strategy for large-scale procedural content generation, making it well-suited for real-time applications in 3D environments.

V. CONCLUSION

Structuring dungeon areas as concentric rings around a central point avoids the inflexible, grid-based limitations of classical algorithms, providing a dynamic and visually unique solution for 2D and 3D worlds. This technique is strong at balancing computational simplicity with artistic flexibility: utilizing direct polar-to-Cartesian (and spherical-to-Cartesian) conversions, it has time complexity in room placement, allowing real-time use without compromising structural integrity or design influence. Designers can readily scale the complexity of dungeons through varying ring numbers, maximum radius, and randomness degree without having to repeat any dungeon generated since each will be unique while maintaining logical connectivity and progression. The concentric ring design inherently facilitates layered gameplay and narrative architecture. Inner rings may serve as safe areas or tutorial spaces, and outer rings bring with them new threats, foes, or thematic content, channelling players through a spatial and narrative experience. This structure aids in pacing and difficulty tuning, imbuing each play session with a sense of adventure and exploration lacking in fixed rectilinear designs. Adaptability is another characteristic of this algorithm. It accommodates hand-tuned as well as completely procedural setups, with precise control over the number of rooms, sizes, and angular positioning within each ring. The central hub and the ability to use three-dimensional room distribution enhance the potential for creativity, making the approach viable for varied game genres and design goals.

Overall, the Radial-Layered method presents a strong, efficient, and highly customizable system for procedural dungeon generation. It enables designers to generate dungeons that are computationally valid and full of gameplay, storytelling depth, and aesthetic interest. As procedural generation continues to be more and more influential in game development, techniques that blend mathematical rigor with artistic freedom like this one are poised to define the next wave of interactive, replayable game worlds.

VI. ACKNOWLEDGMENT

We would like to thank Dayananda Sagar University for allowing us the privilege and resources to work on this project under our Bachelor's degree program. We offer our sincere thanks to Dr. Udaya Kumar Reddy K R, Dean, School of Engineering & Technology, Dayananda Sagar University, for his constant support and guidance. We are particularly grateful to Dr. Girisha GS, Chairman, Department of Computer Science and Engineering, for the academic atmosphere and support required for the successful completion of this work. We are highly grateful to our project guide, Mr. Prolay Biswas Professor, Department of Computer Science and Engineering, for his valuable guidance, helpful criticism, and unstinting support in every step of the project. Lastly, we thank our families and friends for continuous encouragement, support, and understanding throughout the duration of this project.

REFERENCES

- [1] M. Hendrikx, S. Meijer, J. Van Der Velden, and A. Iosup, "Procedural Content Generation for Games: A Survey," *ACM Transactions on Multimedia Computing, Communications, and Applications*, vol. 9, no. 1, pp. 1–22, Feb. 2013.
- [2] J. Togelius, G. N. Yannakakis, K. O. Stanley, and C. Browne, "Procedural Content Generation: Goals, Challenges and Actionable Steps," in *Artificial and Computational Intelligence in Games*, Dagstuhl Follow-Ups, vol. 6, pp. 61–75, 2015.
- [3] A. Shaker, N. Shaker, and J. Togelius, *Procedural Content Generation in Games: A Textbook and an Overview of Current Research*. Springer, 2016.
- [4] M. Cook, S. Colton, A. Raad, and J. Gow, "Mechanic Miner: Reflection-Driven Game Mechanic Discovery and Level Design," in *IEEE Transactions on Games*, vol. 11, no. 3, pp. 261–273, 2019.
- [5] K. Smith, J. Whitehead, and M. Mateas, "Tanagra: A Mixed-Initiative Level Design Tool," in *Proc. Fifth Int. Conf. on the Foundations of Digital Games*, pp. 209–216, 2010.
- [6] G. N. Yannakakis and J. Togelius, "Experience-Driven Procedural Content Generation," *IEEE Trans. Affective Computing*, vol. 2, no. 3, pp. 147–161, Jul.–Sep. 2011.
- [7] Balint, J. Timothy, and Rafael Bidarra. "Procedural generation of narrative worlds." *IEEE Transactions on Games* 15, no. 2 (2022): 262-272.
- [8] Lazaridis, Lazaros, Konstantinos-Filippos Kollias, George Maraslidis, Heraklis Michailidis, Maria Papatismouli, and George F. Fragulis. "Auto Generating Maps in a 2D Environment." In *International Conference on Human-Computer Interaction*, pp. 40-50. Cham: Springer International Publishing, 2022..
- [9] K. Compton and M. Mateas, "Procedural Level Design for Platform Games," in *Proceedings of the Artificial Intelligence and Interactive Digital Entertainment Conference (AIIDE)*, 2006.
- [10] A. Summerville, S. Snodgrass, M. Guzdial, C. Holmgård, A. K. Hoover, A. Isaksen, and G. N. Yannakakis, "Procedural Content Generation via Machine Learning (PCGML)," *IEEE Transactions on Games*, vol. 10, no. 3, pp. 257–270, Sep. 2018.
- [11] J. Dormans, "Adventures in Level Design: Generating Missions and Spaces for Action Adventure Games," in *Proceedings of the 2010 Workshop on Procedural Content Generation in Games*, pp. 1–8, ACM, 2010.
- [12] J. Togelius and J. Schmidhuber, "An Experiment in Automatic Game Design," in *IEEE Symposium on Computational Intelligence and Games*, pp. 111–118, 2008.
- [13] Liapis, Antonios, Héctor P. Martínez, Julian Togelius, and Georgios N. Yannakakis. "Adaptive game level creation through rank-based interactive evolution." In *2013 IEEE Conference on Computational Intelligence in Games (CIG)*, pp. 1-8. IEEE, 2013.
- [14] Huijser, Remco, Jeroen Dobbe, Willem F. Bronsvort, and Rafael Bidarra. "Procedural natural systems for game level design." In *2010 Brazilian Symposium on Games and Digital Entertainment*, pp. 189-198. IEEE, 2010.

- [15] P. Mawhorter and M. Mateas, "Procedural Level Design for Story: A Mixed-Initiative Approach," in Proceedings of the FDG Workshop on Procedural Content Generation, 2010.